AI-Auditor: A Data Auditing Framework for Enhancing the Trustworthiness of AI Models

Lipeng Wang, Mingsheng Hu, Laurence T. Yang, Xinfang Sun, and Zhong Chen

***Abstract*—Artificial intelligence (AI) is now widely adopted across fields, prompting AI companies to deploy models to the cloud for cost, resource, and scalability benefits. However, these cloud-hosted models face security and credibility challenges. Equipment failures or network attacks may compromise data integrity, while commercial interests might cause AI companies or cloud providers to deploy models deviating from their stated specifications, such as version or copyright details. To address these issues, we introduce AI-auditor, a novel data auditing framework that verifies both data integrity and model alignment with declared specifications. Using a challenge-response approach, AI-auditor maintains constant bandwidth usage and O(1) verification efficiency, regardless of file size. It also features a multikey management server mechanism to generate user keys, minimizing risks of single-point failures and trust issues. Analysis and simulations confirm AI-auditor's correctness, security, and high execution efficiency.**

***Index Terms*—Artificial intelligence (AI), cloud storage, data auditing, data integrity, trustworthiness.**

NOMENCLATURE

Notations	Descriptions
q	Large prime number.
G_1, G_2	Multiplicative cyclic groups.
e	Bilinear pairing operation.
g	Generator of G_1.
H_1, H_2	Cryptographic hash functions.
$Sig()$	Identity-based signature function.
$idsk$	Secret key for the $Sig()$ function.
ID	User identity.
KS	Key management server.
σ	System private key.
D	System public key.
γ	Private key of KS.
P	Public key of KS.
F	Outsourced data model.
name	Identifier of the model F.
m_i	ith block of the model F.
RH	Hash value of the model's metadata.
b	User's private key.
A	User's public key.
Λ	Authenticator set for the model F.
tag	File tag of the model F.
CHAL	Challenge computed by TPA.
proof	Proof generated by the cloud.

Received 1 March 2025; revised 26 April 2025; accepted 16 July 2025. Date of publication 18 September 2025; date of current version 5 November 2025. This work was supported in part by the Outstanding Young Science and Technology Talent Project of Zhengzhou (Key Technologies for Data Security Protection in Off-chain Collaboration), in part by the Primary Research and Development Plan of Jiangsu Province under Grant BE2023025, Grant BE20230255, and Grant QNGG-232239, and in part by the Special Funding for the Construction of the Young Backbone Teacher Training Project of Zhengzhou Normal University. Paper no. TII-25-1091. *(Corresponding author: Laurence T. Yang.)*

Lipeng Wang, Mingsheng Hu, and Xinfang Sun are with the School of Information Science and Technology, Zhengzhou Normal University, Zhengzhou 450044, China (e-mail: humingsheng@zznu.edu.cn; hy@stu.zznu.edu.cn).

Laurence T. Yang is with the School of Computer Science and Artificial Intelligence, Zhengzhou University, Zhengzhou 450000, China (e-mail: wlpscu@pku.edu.cn; ltyang@ieee.org).

Zhong Chen is with the School of Computer Science, Peking University, Beijing 100871, China (e-mail: zhongchen@pku.edu.cn).

Digital Object Identifier 10.1109/TII.2025.3593981

I. INTRODUCTION

ARTIFICIAL intelligence (AI) is the ability of machines to replicate human intelligence, allowing them to undertake tasks, such as learning, problem-solving, reasoning, perception, and comprehending natural language. AI systems can utilize algorithms and vast datasets to analyze information, make predictions, and streamline operations. In modern society, AI plays a vital role across various fields. For example, in healthcare, AI assists in early disease detection, tailored treatment, and pharmaceutical development. By fostering innovation and tackling global issues, AI has established itself as an essential driver of societal advancement.

The advancement of AI requires a strong emphasis on security and trustworthiness to ensure its ethical and responsible use. AI companies increasingly rely on cloud infrastructure to host AI models, offering services to external users. This cloud-based approach allows models to be stored on remote servers, accessible and manageable via the internet, bypassing the limitations of local storage and enabling seamless data access for users. However, despite these advantages, cloud hosting introduces two critical challenges. First, once an AI model is deployed to a cloud server, the model owner loses direct control, exposing it to risks of data corruption from hardware or software failures, or cyberattacks, which makes preserving the model's integrity more difficult. Second, driven by commercial interests or reputation management, AI operators or cloud service providers may deploy models that diverge from their stated specifications—such as version numbers or copyright details. These discrepancies

1941-0050 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: James Cook University of Northern Queensland. Downloaded on October 10,2026 at 04:01:53 UTC from IEEE Xplore. Restrictions apply.

erode the credibility and reliability of AI models, posing significant trust concerns. For instance, in early 2025, Microsoft and OpenAI initiated an investigation into whether DeepSeek, a Chinese AI company, illicitly accessed data outputs from OpenAI's technology [1]. Proving the legitimacy of its intellectual property, while protecting sensitive model data remains one of DeepSeek's most effective strategies for responding to this investigation.

To address these aforementioned security and trustworthiness challenges, one approach involves a trusted third party retrieving AI models from the cloud and verifying their content against metadata, such as hash values, to confirm alignment with declared specifications. This third party must also safeguard against unauthorized data access. However, this method requires transferring the entire model over the network, resulting in substantial bandwidth usage and an increased risk of data leakage [2]. Recently, researchers have explored trusted execution environments (TEEs) to enhance the privacy and integrity of AI models [3], [4]. TEEs offer hardware-level isolation, preventing cloud providers from accessing model parameters, while ensuring data integrity through remote attestation. Nevertheless, existing TEE solutions, such as Intel SGX, provide limited secure memory (e.g., SGX v1 supports only 128 MB of EPC), constraining their adaptability of AI models across different applications. Moreover, TEEs remain vulnerable to security threats, such as side-channel attacks [5].

To verify AI model credibility more efficiently, we propose AI-auditor, a novel data audit framework based on the provable data possession (PDP) scheme. PDP, a cryptographic technique, enables remote verification of outsourced data integrity without full downloads [6], [7]. Built on our provably secure PDP variant, AI-auditor allows auditors to confirm integrity without holding the data, minimizing network transmission. This delivers low bandwidth usage and robust privacy protection. In addition, AI-auditor achieves $O(1)$ verification efficiency, unaffected by dataset size. Unlike traditional PDP relying on a single key management center—prone to attacks and failures—AI-auditor employs multiple key management servers (KMSs) to distribute subprivate keys, ensuring security even if some keys are compromised and eliminating single point of failure. Beyond integrity, AI-auditor leverages metadata to validate model credibility, confirming compliance with advertized specifications, such as version numbers and copyrights. Notably, integrity alone does not ensure metadata consistency, as an AI operator may upload model files to the cloud that mismatch their claimed metadata, compromising the file's credibility despite intact content. The system model of AI-auditor is illustrated in Fig. 1.

The contributions of this study are as follows.

1) We present AI-auditor, a novel AI model auditing framework designed to verify data integrity and ensure alignment with their declared specifications, enhancing their credibility. AI-auditor operates with minimal bandwidth and achieves $O(1)$ execution efficiency.
2) We introduce a novel multikey management server mechanism for AI-auditor. Even if some servers are compromised or fail, AI-auditor remains operational, effectively preventing single points of failure and trust.

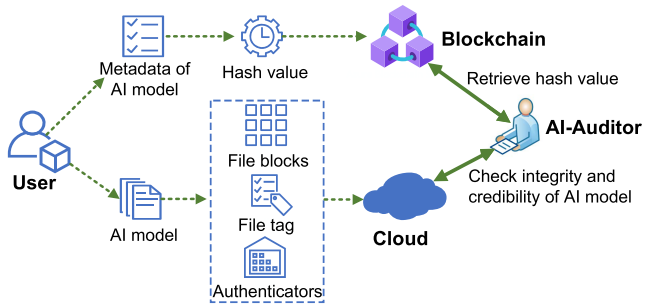


Fig. 1. System model of AI-auditor.

3) We analyzed the correctness and security of AI-auditor, and evaluated its performance through experimental simulations, demonstrating audit soundness, detectability, and high execution efficiency.

The rest of this article is organized as follows. Section II reviews related work. Section III introduces essential notations and foundational concepts. Section IV details the system model, including architecture, design goals and algorithm model. Section V describes the technical implementation and analyzes its correctness. Section VI formally proves AI-auditor's security. Section VII provides a comprehensive performance evaluation. Finally, Section VIII concludes this article.

II. Related Work

Currently, two primary approaches exist for verifying AI model credibility. The first employs cryptographic techniques, such as hash functions [8], multiparty computation (MPC) [9], homomorphic encryption (HE) [10], and differential privacy [11]. The second leverages TEE-based methods, including Intel SGX and TrustZone [3], [12], [13]. Cryptography methods offer robust security and flexibility, allowing customization for diverse applications. However, these methods typically suffer from low execution efficiency. In contrast, TEE-based methods provide hardware-level isolation, shielding AI models from external interference without the complex context switching and instruction simulation required by virtual machines (VMs) or containers, thereby improving performance [14], [15]. However, TEEs are constrained by small secure memory capacities, with excess usage triggering costly paging operations. This makes TEEs suitable for small-scale computations but less effective for large-scale tasks, such as machine learning or other computationally intensive tasks.

To overcome the limitations of traditional cryptographic and TEE-based auditing solutions, we propose AI-auditor, which is based on an enhanced PDP scheme. This PDP scheme eliminates the need for auditors to store audited data, ensuring bandwidth usage independent of data size, and delivers high verification efficiency. Key management is an essential aspect of PDP schemes, directly impacting both security and usability. Early PDP schemes required users to generate and maintain their own public and private keys [6], [16], [17], [18]. However, the loss of these keys prevented users from uploading or verifying data, posing significant challenges. To enable multiuser

Authorized licensed use limited to: James Cook University of Northern Queensland. Downloaded on October 10,2026 at 04:01:53 UTC from IEEE Xplore. Restrictions apply.

collaboration, public key infrastructure (PKI) was integrated into PDP schemes [19], [20], [21]. While PKI enables secure key distribution, it introduces complexity by requiring users to verify certificates during data uploads or validation. To address this issue, researchers proposed identity-based PDP schemes, allowing users to employ their identities (e.g., email addresses) as public keys [22], [23], [24]. These methods simplify key management by eliminating the certificate-related overheads, such as issuance, storage, distribution, and revocation. In addition, identity-based PDP schemes offer enhanced flexibility for applications such asgroup encryption, broadcast encryption, and secure multicast communication. However, they still face the key escrow problem, where a trusted authority retains access to private keys, undermining security. To address this issue, researchers developed certificateless PDP schemes [25], [26], [27], [28], [29]. In this approach, users collaborate with a key generation center (KGC) to generate public and private keys, ensuring the KGC cannot independently access users' private keys, thus enhancing security. Certificateless PDP schemes also reduce the computational and communication overhead associated with certificate management (e.g., issuance, verification, and revocation), improving overall efficiency. Despite these advancements, PKI-based, identity-based, and certificateless PDP schemes all rely on a centralized key management center to generate user keys, which introduces potential single points of failure and trust.

When performing data auditing, it is essential to distinguish between public and private auditing modes, which define the interaction between the data owner and the auditor. Public auditing permits an independent third party to verify data [24], [25]. In contrast, private auditing confines verification to the data owner or a designated entity [19], [30]. Given their wider applicability in real-world scenarios, algorithms that support public auditing are the focus of this article. Zhang et al. [31] introduced FL-auditor, a public data auditing method to counter Byzantine attacks in federated learning. It utilizes a third-party auditor to sample and assess participant trustworthiness, thereby strengthening the global model's resilience against such attacks. Wu et al. [32] proposed a public auditing scheme for cloud-stored data shared among large user groups, while ensuring user anonymity . Current public auditing framework based on the PDP algorithm can effectively verify data integrity, but lack robust mechanisms for assessing data credibility. Truong et al. [33] proposed a GDPR-compliant framework based on blockchain, capitalizing on its decentralized and transparent nature to monitor data access and ensure regulatory compliance. In addition, audit results are stored on the blockchain, preventing them from unauthorized tampering. However, blockchain's performance constraints limit the framework's efficiency. In 2023, we proposed a framework that can audit both data integrity and semantic consistency, based on blockchain and the PDP scheme [34]. The audit results can serve as references for subsequent data recovery and accountability. However, frequent blockchain interactions during data uploads and verification reduce efficiency, and reliance on a centralized key management center for key generation introduces risks of single points of failure and trust.

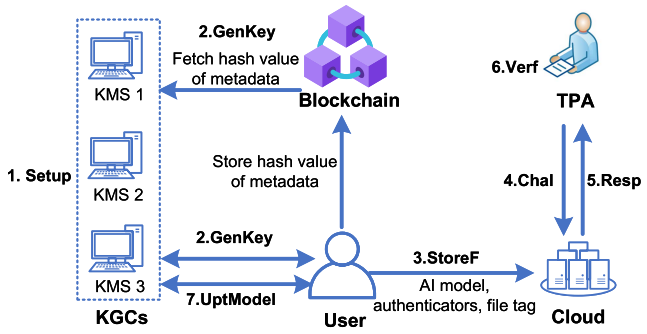


Fig. 2. System framework of AI-auditor.

III. Notations and Preliminaries

A. Notations

For simplicity, we define some symbols in Nomenclature.

B. Bilinear Pairing

Our proposed AI-auditor employs bilinear pairing to verify the correctness of the proof provided by the cloud. The operation satisfies three essential properties: Bilinearity, nondegeneracy, and computability. For a detailed explanation, refer to [23].

C. Computational Hard Problems

To demonstrate the security of AI-auditor, we rely on two computationally difficult problems: The computational Diffie–Hellman (CDH) problem and the discrete logarithm (DL) problem. For a detailed explanation, see [23].

IV. System Model

A. System Framework

The system framework of the proposed AI-auditor is depicted in Fig. 2. AI-auditor comprises five entities: Blockchain, user, KGCs, third-party auditor (TPA), and cloud. When a user, such as an AI model operator, intends to upload a model to the cloud, the model and its associated metadata (e.g., version number, copyright details) must first be verified by a designated number of authoritative institutions and approved through a voting procedure. If verification fail, AI-auditor rejects the upload request. In this system, these authoritative institutions are designated as KMSs, which collectively constitute the KGCs.

AI-auditor begins by executing the *Setup* algorithm to initialize the system environment, during which each KMS generates its public and private keys. Subsequently, the user runs the *GenKey* algorithm to generate their own public and private keys, a process that requires approval via a voting mechanism involving a quorum of KMSs. This voting approach mitigates single point of failure and strengthens resistance against KMS manipulation. After key generation, the user executes the *StorF* algorithm to upload the model to the cloud, where the model is segmented into fixed-size blocks and distributed across multiple cloud storage devices. Upon successful upload, the AI model is deployed and ready to deliver services. To verify the model's

Authorized licensed use limited to: James Cook University of Northern Queensland. Downloaded on October 10,2026 at 04:01:53 UTC from IEEE Xplore. Restrictions apply.

integrity and consistency with its metadata, the TPA executes the *Chal* algorithm to issue a challenge to the cloud. The cloud responds by executing the *Resp* algorithm, generating a proof from the stored model and returning it to the TPA. The TPA then validates the proof through running the *Verf* algorithm to ensure that the model's content is intact and aligns with its metadata. In addition, AI-auditor allows the user to update model metadata using the *UptModel* algorithm, maintaining its accuracy and relevance. The subsequent sections elaborate on the five entities within AI-auditor.

User: The user, typically an AI model operator, uploads their model to the cloud to secure data and, upon successful deployment, can generate revenue. To bolster security, the operator must provide credentials confirming that the cloud-hosted model matches its advertized specifications, such as version number, copyright details, and other pertinent attributes.

KGCs: KGCs comprise a group of KMSs that collectively vote to authorize a user's model upload or update. This voting mechanism reduces the risk of KMS manipulation.

Blockchain: The blockchain stores the hash value of model metadata, safeguarding its integrity and preventing unauthorized alterations. In AI-auditor, it may be implemented as either a consortium or public blockchain.

TPA: The TPA verifies the integrity of the cloud-hosted model and its consistency with the associated metadata. AI-auditor's support for public auditing allows other entities, including users, to serve as the TPA.

Cloud: The cloud consists of a group of distributed storage devices that offer near-unlimited capacity. However, factors, such as equipment failures or hacker attacks, can compromise the integrity of cloud-hosted models. For reasons, such as maintaining business reputation, the cloud may fabricate proofs in response to challenges issued by the TPA in an attempt to pass the TPA's verification.

B. Design Goals

The proposed AI-auditor aims to ensure model integrity and confirm consistency with associated metadata. However, it faces several security challenges. First, if models are corrupted, cloud service providers might forge data to safeguard their reputation and evade TPA verification. Second, during public and private key generation, KGCs could exploit vulnerabilities to compromise users' private keys. Third, users driven by commercial gain might intentionally upload models that deviate from their declared specifications. To mitigate these threats, AI-auditor must fulfill the following objectives.

1) *Support for updating model metadata:* AI-auditor enables users to update the metadata associated with their models, provided they have the necessary authorization.
2) *Correctness:* When all entities in AI-auditor adhere to established procedures, responses generated by the cloud from locally stored models can successfully pass TPA verification.
3) *Auditing soundness:* If a cloud-hosted AI model is corrupted, the cloud cannot produce a response that satisfies TPA verification.
4) *Public auditing:* AI-auditor enables any entity that does not possess the model owner's private key to conduct data audits.

C. Algorithm Model

Our proposed AI-auditor comprises seven algorithms: Setup, GenKey, StorF, Chal, Resp, Verf, and UptModel. This section outlines the purpose of each algorithm.

Setup: Each key management center within the KGCs is responsible for initializing its environmental parameters, generating its own public and private keys, and collaborating with other key management centers to generate the system's public and private keys.

GenKey: When users upload models to the cloud, the algorithm requires approval from a quorum of key management centers to generate the user's public and private keys, ensuring no individual key management center can independently derive the user's private key.

StorF: This algorithm enables model uploads to the cloud, generating the corresponding file authentication set and file tag, which are stored with the model.

Chal: After a model is uploaded to the cloud, the TPA issues a challenge to the cloud to verify the model's data integrity and consistency with its specifications.

Resp: In response to the challenge, the cloud computes a proof using the stored model and returns it to the TPA.

Verf: The TPA evaluates the proof from the cloud. Successful verification confirms that the model's content remains intact and aligns with its specifications.

UptModel: This algorithm enables the user and KGCs to jointly update the model's metadata, requiring consensus from all KMSs for approval.

V. Proposed Scheme

A. Description of AI-auditor

In the proposed AI-auditor framework, the user divides the outsourcing model F into n blocks, represented as $\{m_1, \ldots, m_i, \ldots m_n\}$, where each $m_i \in \mathbb{Z}_q^*$ and q is a large prime number. In addition, the user computes the hash value RH of the model's metadata, comprising the model's version number and copyright details, and then records it on the blockchain. For dynamic data operations, such as updating the metadata of model F, AI-auditor also plays a pivotal role in auditing model integrity and credibility of the model.

The KGCs comprise d KMSs, represented as $\{KS_1, KS_2, \ldots, KS_d\}$, each with a unique identity $\{SD_1, SD_2, \ldots, SD_d\}$, where $SD_i \in \mathbb{Z}_q^*$ and $1 \leq i \leq d$. When executing the *GenKey* algorithm, consensus from at least t KMSs $(t \leq d)$ is required for the user to generate their public and private keys.

1) *Setup:* The KMSs initialize the system by generating the system's public and private keys along with public parameters. The process is outlined as follows.
   i) KGCs select a bilinear map $e : G_1 \times G_1 \to G_2$, where G_1 and G_2 are two multiplicative cyclic

Authorized licensed use limited to: James Cook University of Northern Queensland. Downloaded on October 10,2026 at 04:01:53 UTC from IEEE Xplore. Restrictions apply.

groups with prime order q, and g is the generator of G_1.

ii) KGCs define two hash functions: $H_1 : \{0,1\}^* \times G_1 \times Z_q^* \to Z_q^*$ and $H_2 : \{0,1\}^* \times G_1 \to Z_q^*$. Here, H_1 is used to generate the user's private key and file tag, while H_2 assists in generating the user's private key. To ensure the integrity of the file tag, KGCs also select a secure ID-based digital signature function, denoted as $Sig()$. In this context, $idsk$ represents the secret key, and ID indicates the user's identity.

iii) Each key server KS_i randomly generates a polynomial of degree $(t-1)$, expressed as $f_i(x) = c_{i,0} + c_{i,1}x + \cdots + c_{i,t-1}x^{t-1}$, where $c_{i,j} \in Z_q^*$, $1 \le i \le d$, and $0 \le j \le t-1$. Here, $c_{i,0}$ serves as the sub-secret of KS_i.

iv) Each KS_i calculates d elements $\{f_i(SD_1), f_i(SD_2), \ldots, f_i(SD_d)\}$ and securely transmits each $f_i(SD_j)$ to the corresponding key management center KS_j through a secret channel.

v) Each KS_i derives its private key as $\gamma_i = \sum_{j=1}^{d} f_j(SD_i)$, and its public key as $P_i = g^{\gamma_i}$.

vi) The system private key is computed as $\sigma = \sum_{i=1}^{d} c_{i,0}$, and the system public key is calculated as $D = g^{\sigma}$.

vii) KGCs publish the public parameters: $\langle e, D, \{P_1, P_2, \ldots P_d\}, G_1, G_2, q, g, H_1, H_2, Sig() \rangle$.

2) *GenKey*: Before uploading files, the user must execute this algorithm to generate their own public and private keys, working with the KGCs throughout the process. For simplicity, we denote the user's identity as ID. The steps are as follows.

i) To obtain the public key A, the user should select a random number a and computes $A = g^a$. The user, then, retrieves RH, corresponding to the metadata of the model F, from the blockchain and sends $\langle ID, A \rangle$ to the d KMSs.

ii) Each key server KS_i retrieves RH from the blockchain, and checks whether ID and RH satisfy the specified requirements. If they do not, the algorithm terminates.

iii) Each KS_i selects a random number $r_i \in Z_q^*$ and computes $B_i = g^{r_i}$ and $X_i = r_i + \gamma_i H_1(ID, A, RH) + H_2(ID, A^{\gamma_i})$. KS_i, then, sends B_i and X_i to the user.

iv) Upon receiving B_i and X_i, the user verifies the data using the equation $g^{X_i} = B_i {P_i}^{H_1(ID,A,RH)} g^{H_2(ID,{P_i}^a)}$. If the verification fails, each KS_i must resend B_i and X_i. Otherwise, the user calculates $b_i = X_i - H_2(ID, {P_i}^a)$

v) The user successfully collects t copies of b_i, denoted as $B = \{b_{i_1}, b_{i_2}, \ldots, b_{i_t}\}$, with the corresponding index set $I = \{i_1, i_2, \ldots, i_t\}$.

vi) The user computes $b = \sum_{\xi=i_1}^{i_t} (b_\xi \rho_\xi) + a$, where $\rho_\xi = \prod_{j \in I, SD_j \neq SD_\xi} \frac{SD_j}{SD_j - SD_\xi}$. The user, then, obtains their private key b and public key A.

3) *StorF:* The user uploads the model file F to the cloud by executing this algorithm. For simplicity, the file's unique identifier is denoted as $name$. The user must compute the file authenticators and file tag corresponding to F and then, upload them along with F to the cloud.

i) The user calculates the file authenticator $T_i = g^{m_i b}$ for each data block m_i, and thereby obtains the file authenticator set $\Lambda = \{T_i\}_{1 \le i \le n}$ corresponding the file F.

ii) The user should calculate Z with the equation $Z = \prod_{\xi=i_1}^{i_t} B_\xi^{\prod_{j \in I, SD_j \neq SD_\xi} \frac{SD_j}{SD_j - SD_\xi}} D^{H_1(ID,A,RH)} A$.

iii) The user should calculate the signature $sig = Sig_{idsk}(ID||name||n||Z)$ and derives the file tag $tag = \{ID||name||n||Z||sig\}$. Afterward, the user sends $\{tag, F, \Lambda\}$ to the cloud.

iv) The cloud uses ID and $\{tag, F, \Lambda\}$ to verify the user's authorization for outsourcing data. If the user is authorized, the cloud checks the validity of the signature $Sig_{idsk}(ID||name||n||Z)$ using ID. Upon successful signature verification, the cloud stores $\{tag, F, \Lambda\}$; otherwise, it denies the user's outsourcing request. Algorithm 1 outlines the process for outsourcing files to the cloud.

4) *Chal:* To verify the model file F stored in the cloud, the TPA computes a challenge and sends it to the cloud to obtain the associated proof.

i) TPA retrieves the file tag tag associated with the model $name$ and the user ID from the cloud. If this step fails, the algorithm terminates.

ii) Upon obtaining tag, the TPA verifies the validity of the signature $Sig_{idsk}(ID||name||n||Z)$ using ID. If the signature check fails, the algorithm terminates.

iii) The TPA creates a random set Q, where $|Q| = c$ and $Q \in \{1, 2, \ldots, n\}$. Next, the TPA constructs a challenge $CHAL = \{i, v_i\}_{i \in Q}$ tied to the model file F, chooses a random number $\upsilon \in Z_q^*$, and computes $C = g^{\upsilon}$.

iv) The TPA delivers υ along with $CHAL$ to the cloud.

5) *Resp:* When the TPA issues a challenge, the cloud leverages the model file data to construct a proof, which it then transmits back to the TPA.

i) The cloud computes $\rho = \sum_{i \in Q} m_i v_i$ with $CHAL$.

ii) The cloud computes $\sigma' = \sigma^{\upsilon} = \prod_{i \in Q} {T_i}^{\upsilon v_i}$ with $CHAL$ and υ.

iii) The cloud transmits the proof $proof = \langle \rho, \sigma' \rangle$ to the TPA for verification.

6) *Verf:* The TPA assesses the cloud's response to confirm the integrity of the stored AI model file and its consistency with the associated metadata.

i) After obtaining the proof $proof = \langle \rho, \sigma' \rangle$ from the cloud, TPA checks the equation $e(\sigma', g) = e(Z, C^{\rho})$. If this validation fails, it indicates that the cloud AI model is either corrupted or does not align with the specified metadata.

7) *UptModel:* In this algorithm, the user collaborates with KGCs to update the metadata of model F. Assume the

Authorized licensed use limited to: James Cook University of Northern Queensland. Downloaded on October 10,2026 at 04:01:53 UTC from IEEE Xplore. Restrictions apply.

hash value of the new model's metadata is RH', and the user with identity ID has outsourced RH' to the blockchain.

i) The user ID submits an update request, including $\langle ID, A\rangle$, to the d KMSs.

ii) Upon receiving $\langle ID, A\rangle$, each key server KS_i verifies whether the user ID is authorized to run the **UptModel** algorithm. If not, the program terminates. Otherwise, KS_i selects a random number ${r_i}' \in Z_q^*$ and computes ${B_i}' = g^{{r_i}'}$ and ${X_i}' = {r_i}' + \gamma_i H_1(ID, A, RH') + H_2(ID, A^{\gamma_i})$. Subsequently, KS_i sends ${X_i}'$ and ${B_i}'$ to the user ID.

iii) Upon receiving ${X_i}'$ and ${B_i}'$, the user verifies their correctness using the equation $g^{{X_i}'} = {B_i}' P_i^{H_1(ID,A,RH')} g^{H_2(ID,{P_i}^a)}$. If the verification fails, the process terminates. Otherwise, the user computes ${b_i}' = {X_i}' - H_2(ID, {P_i}^a)$. The user then collects t copies of ${b_i}'$, denoted as $B = \{{b_{i_1}}', {b_{i_2}}', \ldots, {b_{i_t}}'\}$, with the corresponding index set $I = \{i_1, i_2, \ldots, i_t\}$.

iv) The user computes $b' = \sum_{\xi=i_1}^{i_t} ({b_\xi}'\rho_\xi') + a$, where $\rho_\xi' = \prod_{j\in I, SD_j \neq SD_\xi} \frac{SD_j}{SD_j - SD_\xi}$. This yields the user's new private key b' and public key A.

v) The user computes the new file authenticator ${T_i}' = g^{m_i b'}$ for each data block m_i stored in the cloud, assembling the new file authenticator set $\Lambda' = \{{T_i}'\}_{1\le i\le n}$.

vi) The user should calculate the new Z' with $Z' = \prod_{\xi=i_1}^{i_t} {B_\xi}' \prod_{j\in I, SD_j\neq SD_\xi} \frac{SD_j}{SD_j - SD_\xi} D^{H_1(ID,A,RH')} A$, the new sig' with $sig' = Sig_{idsk}(ID||name||n||Z')$ and $tag' = \{ID||name||n||Z'||sig'\}$.

vii) Finally, the user updates the cloud data from $\{F, tag, \Lambda\}$ to $\{F, tag', \Lambda'\}$.

B. Correctness Analysis

We first establish the validity of the equation $b = \Delta + \sigma H_1(ID, A, RH) + a$, where $\Delta = \sum_{\xi=i_1}^{i_t} (r_\xi \prod_{j\in I, SD_j\neq SD_\xi} \frac{SD_j}{SD_j - SD_\xi})$.

Proof: Due to page constraints, the proof is provided in the appendix.

Next, we demonstrate that the verification equation $g^{X_i} = B_i P_i^{H_1(ID,A,RH)} g^{H_2(ID,{P_i}^a)}$ in the **GenKey** algorithm holds.

Proof: Due to page constraints, the proof is provided in the appendix.

Finally, we prove that the verification equation $e(\sigma', g) = e(Z, C^\rho)$ in the **Verf** algorithm remains valid.

Proof: Due to page constraints, the proof is provided in the appendix.

VI. Security Analysis

Theorem 1 (Auditing Soundness): In our proposed solution, AI-auditor, when the AI model file stored in the cloud is corrupted or inconsistent with the designated metadata, the cloud is unable to produce a legitimate proof that satisfies the TPA's validation process.

Proof: In light of page constraints, the proof is provided in the appendix.

Theorem 2 (Detectability): We assume that the AI model file stored in the cloud is segmented into n data blocks. Due to equipment failure or network attacks, π of these n blocks are corrupted. In the AI-auditor system, when the TPA randomly selects c blocks for verification, the probability of identifying at least one corrupted block is given by $1 - (\frac{n-\pi}{n})^c$.

Proof: Due to page constraints, the proof follows a method similar to that in [6] and is not detailed here. The results indicate that AI-auditor can detect file block corruption with a specific probability by challenging a fixed number of blocks, independent of the total number of blocks audited. For example, with n file blocks and $\pi = 0.01n$ corrupted, the detection probability is given by $1 - (\frac{n-\pi}{n})^c = 1 - (\frac{n-0.01n}{n})^c = 1 - 0.99^c$. Setting $c = 460$ yields $1 - 0.99^{460} \ge 99\%$, indicating that AI-auditor needs to challenge only 460 blocks to detect anomalies with at least 99% probability, regardless of n. Thus, AI-auditor achieves $O(1)$ execution efficiency in the data verification process.

Algorithm 1: The Procedure of Outsourcing AI Models to the Cloud.

Input: the model F, the file tagtag, the file authenticator set Λ, the authorized user identify set L, the dataset stored in key-value form DS.
Output: $ret \in \{true, false\}$

```
ret ← false
(ID, name, n, Z, sig) ← parse(tag)
// Check whether the user ID is authorized.
if L.search(ID) == NULL then
    return ret
// Verify the signature within the file tag.
ret ← checkSig(ID, name, n, Z, sig)
if ret == false then
    then ret
// Construct a storage index for the model F.
Index ← constrIndex(ID, name)
// Check if the model F is already stored in the cloud.
if DS.search(Index) ≠ NULL then
    return ret
// Store {tag, F, Λ} to the cloud.
DS.store(Index, {tag, F, Λ})
ret ← true
return ret
```

VII. Evaluations

To gauge the effectiveness of AI-auditor, we evaluated its performance on a personal computer with the following experimental setup: CPU: 12th Gen Intel i5-12500H 2.5 GHz; RAM: 16.0 GB; Operating system: Windows 11; Programming language: Java 22; Blockchain framework: Quorum 2.0. It is worth noting that network latency, affected by various external factors, was excluded from the efficiency evaluation of AI-auditor.

Authorized licensed use limited to: James Cook University of Northern Queensland. Downloaded on October 10,2026 at 04:01:53 UTC from IEEE Xplore. Restrictions apply.

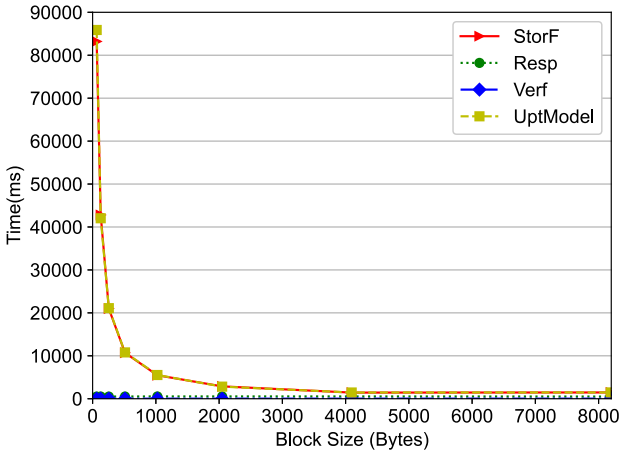


Fig. 3. Determination of the optimal block size.

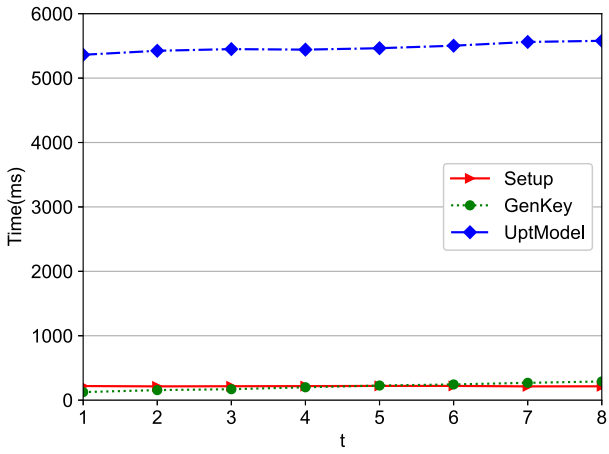


Fig. 4. Execution time of the three algorithms as a function of t.

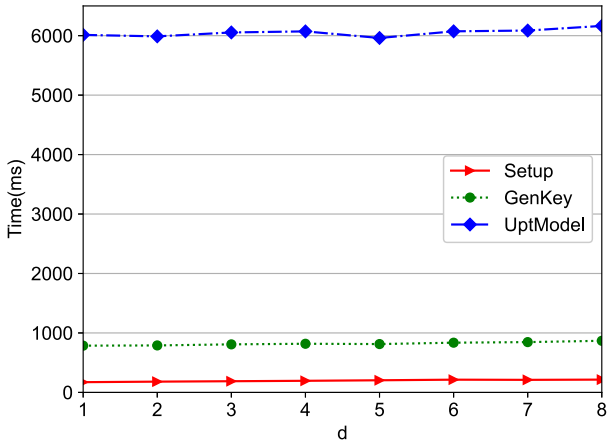


Fig. 5. Execution time of the three algorithms as a function of d.

In the *GenKey* algorithm, AI-auditor retrieves the hash value of the AI model's metadata from the blockchain. This value is fetched locally from the blockchain node without engaging the blockchain consensus mechanism, meaning that the consensus protocol has a minimal impact on the execution of GenKey. In the *UptModel* algorithm, the user must first update the hash value of the AI model's metadata and upload it to the blockchain. However, the time required for this upload is not included in the execution time of the *UptModel* algorithm.

To boost the efficiency of AI-auditor, we investigated the optimal file block size, as it significantly affects the execution efficiency of four algorithms: StorF, Resp, Verf, and UptModel. For our tests, we fixed the file size at 1 MB, designated 100 file blocks as the challenge set, and applied $d = 3$ and $t = 2$. We analyzed the relationship between file block size and the execution time of these algorithms, with the results presented in Fig. 3. Among the algorithms, StorF and UptModel were the most time-consuming, while Resp and Verf had comparatively negligible time costs. Moreover, the results revealed that as the file block size increases, the execution times of StorF and UptModel initially decrease, but start to rise again when the block size exceeds 4 kb. Based on these findings, we determined that 4 kb is the optimal block size for AI-auditor.

Next, we investigate the relationship between the minimum number of KMSs required for consensus, denoted as t, and the corresponding execution time. In this analysis, we set the total number of KMSs (d) to 8, the number of outsourced file blocks (n) to 1000, and the number of challenged file blocks (c) to 100. We evaluate the execution efficiency of three algorithms: Setup, Genkey, and UptModel. The experimental results are presented in Fig. 4. The findings show that as t increases, the execution time of Genkey and UptModel rise, while the execution time of the Setup algorithm remains constant. This variation arises because Genkey and UptModel require users to collect t copies of b_i.

Third, we analyze the relationship between the total number of KMS (d) and execution time. For this evaluation, we fix t at 8, n at 1000, and c at 100. We assess the execution times of the three algorithms—Setup, Genkey, and UptModel—with results shown in Fig. 5. The findings indicate that as d increases, the execution times of all three algorithms increase, though with some fluctuations observed.

Next, we assess the runtime of each algorithm in AI-auditor to lay the groundwork for future performance optimizations. For this evaluation, we set the file size to 1 MB, the challenged file block count to $n/2$, and d and t set to 3 and 2, respectively. The experimental results are depicted in Fig. 6. The results indicate that StorF, Resp, and UptModel require significant execution time, suggesting a focus on optimizing these three algorithms. Notably, users execute StorF only once during file upload, and UptModel only once when updating the model, meaning that these algorithms have low execution frequence. Consequently, greater attention should be directed toward optimizing the Resp algorithm.

Then, we assess the execution time of three frequently executed algorithms: Chal, Resp, and Verf. For this evaluation, we set the file size to 16 MB, with d set to 3 and t set to 2. The results are displayed in Fig. 7. We observe that the execution times of Chal and Resp increase with the number of challenged file blocks, with Resp taking the longest. Since the data are distributed across multiple storage devices in the cloud, the Resp

Authorized licensed use limited to: James Cook University of Northern Queensland. Downloaded on October 10,2026 at 04:01:53 UTC from IEEE Xplore. Restrictions apply.

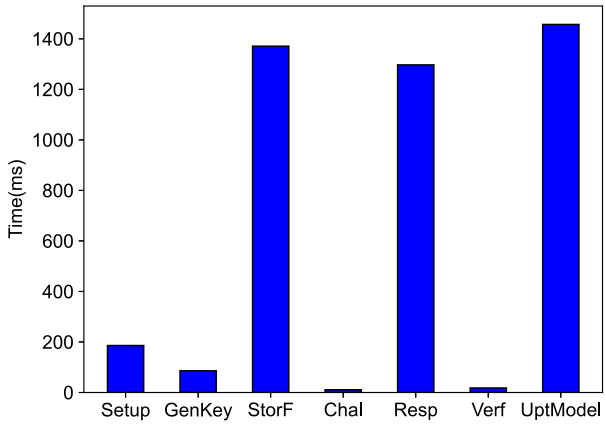


Fig. 6. Runtime of individual algorithms within AI-auditor.

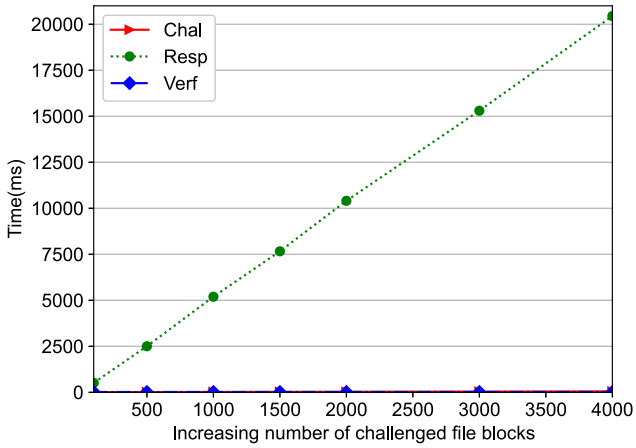


Fig. 7. Execution time of the three frequently executed algorithms.

algorithm can be executed in parallel, significantly reducing the execution time compared to the results shown in Fig. 7.

Finally, we compare our scheme with Deng et al. [29] certificateless provable data possession scheme , focusing on the frequently executed algorithms: Chal, Resp, and Verf. For comparison, we measure the total execution time of these algorithms, setting the file size to 16 MB, $d = 3$, $t = 2$, and the file block size to 4 kb. The results are presented in Fig. 8. For a challenged block count of 4000, Deng et al. [29] scheme requires 22 000 ms, whereas ours takes 20 506 ms, achieving approximately a 7% performance improvement. This enhancement stems from Deng et al. [29] scheme requiring computationally expensive bilinear pairing operations in the Resp algorithm and dual data verification in the Verf algorithm. Furthermore, beyond these performance benefits, our scheme can audit the integrity of AI model data and ensure adherence to specified requirements.

VIII. Conclusion

In this article, we propose AI-auditor, a novel data auditing framework that verifies both the data integrity of AI models and their alignment with the specifications claimed by AI companies. Built on a newly designed PDP scheme, AI-auditor achieves

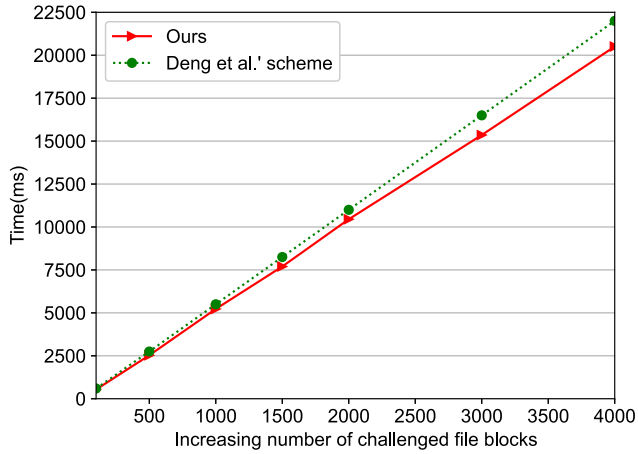


Fig. 8. Comparison of total execution time for three frequently executed algorithms.

minimal bandwidth consumption, while maintaining $O(1)$ execution efficiency. To mitigate risks associated with single points of failure and trust, we incorporated a multikey management server mechanism into AI-auditor, ensuring system resilience even if some servers fail. We evaluated the framework's correctness and security, with simulation experiments confirming its high execution efficiency. AI-auditor is well-suited for applications, such as regulatory supervision. In the future, we plan to further refine its algorithmic efficiency and broaden its range of applications.

References

[1] Reuters, "Microsoft probes if deepseek-linked group improperly obtained openai data," Jan. 30 2025. [Online]. Available: https://www.reuters.com/technology/microsoft-probing-if-deepseek-linked-group-improperly-obtained-openai-data-2025-01-29

[2] G. Han and H. Li, "Sec-auditor: A blockchain-based data auditing solution for ensuring integrity and semantic correctness," *Computers, Mater. Continua*, vol. 80, no. 2, pp. 2121–2137, 2024.

[3] D. Li, Z. Zhang, M. Yao, Y. Cai, Y. Guo, and X. Chen, "TEESlice: Protecting sensitive neural network models in trusted execution environments when attackers have pre-trained models," *ACM Trans. Softw. Eng. Methodol.*, pp. 1–18, 2024.

[4] F. Tramer and D. Boneh, "Slalom: Fast, verifiable and private execution of neural networks in trusted hardware," in *Proc. 7th Int. Conf. Learn. Representations*, New Orleans, USA, 2019, pp. 1–19.

[5] D. Cerdeira, N. Santos, P. Fonseca, and S. Pinto, "SoK: Understanding the prevailing security vulnerabilities in TrustZone-assisted TEE systems," in *Proc. IEEE Symp. Secur. Privacy*, San Francisco, USA, 2020, pp. 1416–1432.

[6] G. Ateniese et al., "Provable data possession at untrusted stores," in *Proc. 14th ACM Conf. Comput. Commun. Secur.*, New York, USA, pp. 598–609.

[7] L. Wang, Z. Guan, Z. Chen, and M. Hu, "Schain: An efficient and secure solution for improving blockchain storage," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 3662–3676, 2023.

[8] R. Goodman and S. Goodman, "Immutable machine learning: Integrating data hashing and retrieval-augmented generation for trustworthy AI," *ResearchGate*, pp. 1–10, 2024. [Online]. Available: https://www.researchgate.net/publication/381401908

[9] C. Juvekar, V. Vaikuntanathan, and A. Chandrakasan, "{GAZELLE}: A low latency framework for secure neural network inference," in *Proc. 27th USENIX Secur. Symp.*, Baltimore, USA, 2018, pp. 1651–1669.

[10] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing, "Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy," in *Proc. Int. Conf. Mach. Learn.*, New York City, USA, 2016, pp. 201–210.

Authorized licensed use limited to: James Cook University of Northern Queensland. Downloaded on October 10,2026 at 04:01:53 UTC from IEEE Xplore. Restrictions apply.

[11] C. Dwork et al., "The algorithmic foundations of differential privacy," *Foundations Trends Theor. Comput. Sci.*, vol. 9, no. 3, pp. 211–407, 2014.
[12] Z. Zhang et al., "No privacy left outside: On the (in-) security of TEE-shielded DNN partition for on-device ML," in *Proc. 2024 IEEE Symp. Secur. Privacy*, San Francisco, USA, 2024, pp. 3327–3345.
[13] A. Paju, M. O. Javed, J. Nurmi, J. Savimäki, B. McGillion, and B. B. Brumley, "SoK: A systematic review of tee usage for developing trusted applications," in *Proc. 18th Int. Conf. Availability, Rel. Secur.*, New York, USA, 2023, pp. 1–15.
[14] B. Ngabonziza, D. Martin, A. Bailey, H. Cho, and S. Martin, "TrustZone explained: Architectural features and use cases," in *Proc. IEEE 2nd Int. Conf. Collaboration Internet Comput.*, Pittsburgh, USA, 2016, pp. 445–451.
[15] W. Zheng et al., "A survey of Intel SGX and its applications," *Front. Comput. Sci.*, vol. 15, pp. 1–15, 2021.
[16] G. Ateniese, R. Di Pietro, L. V. Mancini, and G. Tsudik, "Scalable and efficient provable data possession," in *Proc. 4th Int. Conf. Secur. Privacy Commun. Netowrks*, New York, USA, 2008, pp. 1–11.
[17] R. Curtmola, O. Khan, R. Burns, and G. Ateniese, "MR-PDP: Multiple-replica provable data possession," in *Proc. 28th Int. Conf. Distrib. Comput. Syst.*, Beijing, China, 2008, pp. 411–420.
[18] J. Yu and H. Wang, "Strong key-exposure resilient auditing for secure cloud storage," *IEEE Trans. Inf. Forensics Secur.*, vol. 12, no. 8, pp. 1931–1940, Aug. 2017.
[19] H. Wang, Q. Wang, and D. He, "Blockchain-based private provable data possession," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 5, pp. 2379–2389, Sep./Oct. 2021.
[20] Y. Li, Y. Yu, R. Chen, X. Du, and M. Guizani, "Integritychain: Provable data possession for decentralized storage," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 6, pp. 1205–1217, Jun. 2020.
[21] Y. Xu, J. Ren, Y. Zhang, C. Zhang, B. Shen, and Y. Zhang, "Blockchain empowered arbitrable data auditing scheme for network storage as a service," *IEEE Trans. Serv. Comput.*, vol. 13, no. 2, pp. 289–300, Mar./Apr. 2020.
[22] H. Wang, Q. Wu, B. Qin, and J. Domingo-Ferrer, "Identity-based remote data possession checking in public clouds," *IET Inf. Secur.*, vol. 8, no. 2, pp. 114–121, 2014.
[23] Y. Zhang, J. Yu, R. Hao, C. Wang, and K. Ren, "Enabling efficient user revocation in identity-based cloud storage auditing for shared Big Data," *IEEE Trans. Dependable Secure Comput.*, vol. 17, no. 3, pp. 608–619, May/Jun. 2020.
[24] H. Wang, D. He, J. Yu, and Z. Wang, "Incentive and unconditionally anonymous identity-based public provable data possession," *IEEE Trans. Serv. Comput.*, vol. 12, no. 5, pp. 824–835, Sep./Oct. 2019.
[25] J. Li, H. Yan, and Y. Zhang, "Certificateless public integrity checking of group shared data on cloud storage," *IEEE Trans. Serv. Comput.*, vol. 14, no. 1, pp. 71–81, Jan./Feb. 2021.
[26] L. Zhou, A. Fu, G. Yang, H. Wang, and Y. Zhang, "Efficient certificateless multi-copy integrity auditing scheme supporting data dynamics," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 2, pp. 1118–1132, Mar./Apr. 2022.
[27] D. He, N. Kumar, S. Zeadally, and H. Wang, "Certificateless provable data possession scheme for cloud-based smart grid data management systems," *IEEE Trans. Ind. Informat.*, vol. 14, no. 3, pp. 1232–1241, Mar. 2018.
[28] D. He, N. Kumar, H. Wang, L. Wang, and K.-K. R. Choo, "Privacy-preserving certificateless provable data possession scheme for Big Data storage on cloud," *Appl. Math. Comput.*, vol. 314, pp. 31–43, 2017.
[29] L. Deng, Z. Chen, Y. Ruan, H. Zhou, and S. Li, "Certificateless provable data possession scheme suitable for smart grid management systems," *IEEE Syst. J.*, vol. 17, no. 3, pp. 4245–4256, Sep. 2023.
[30] H. Wang, D. He, A. Fu, Q. Li, and Q. Wang, "Provable data possession with outsourced data transfer," *IEEE Tran. Serv. Comput.*, vol. 14, no. 6, pp. 1929–1939, Nov./Dec. 2021.
[31] Z. Zhang, L. Wu, D. He, J. Li, N. Lu, and X. Wei, "Using third-party auditor to help federated learning: An efficient Byzantine-robust federated learning," *IEEE Trans. Sustain. Comput.*, vol. 9, no. 6, pp. 848–861, Nov./Dec. 2024.
[32] L. Wu, J. Wang, S. Zeadally, and D. He, "Privacy-preserving auditing scheme for shared data in public clouds," *J. Supercomputing*, vol. 74, pp. 6156–6183, 2018.
[33] N. B. Truong, K. Sun, G. M. Lee, and Y. Guo, "GDPR-compliant personal data management: A blockchain-based solution," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 1746–1761, 2019.
[34] L. Wang, Z. Guan, Z. Chen, and M. Hu, "Enabling integrity and compliance auditing in blockchain-based GDPR-compliant data management," *IEEE Internet Things J.*, vol. 10, no. 23, pp. 20955–20968, Dec. 2023.



Lipeng Wang received the B.E. degree in software engineering and B.Sc. degree in computer science and technology from Sichuan University, Chengdu, China, in 2009 and 2012, respectively, and the Ph.D. degree in computer science from Peking University, Beijing, China, in 2023.

He currently serves as the Director of the Software Science Institute with Zhengzhou Normal University, Zhengzhou, China. He has published more than 40 scientific papers, including TIFS, TII, IoTJ, and others. His research interests include cryptography and blockchain.



Mingsheng Hu received the B.S degree in computer software from Central China Normal University, Wuhan, China, in 1997, the M.S. degree in software engineering from Information Engineering University, Zhengzhou, China, in 2005, and the Ph.D. degree in control engineering from the Huazhong University of Science and Technology, Wuhan, in 2007.

Since 1997, he has worked with Zhengzhou Normal University, Zhengzhou, where he is currently a Professor. He has published more than 50 papers. His research interests include information security and complex network.

Dr. Hu was the recipient of the National Science Fund of China.



Laurence T. Yang received the B.E. degree in computer science and technology and the B.Sc. degree in applied physics from Tsinghua University, Beijing, China, in 1992, and the Ph.D. degree in computer science from the University of Victoria, Victoria, BC, Canada, in 2006.

He is currently a Professor with the School of Computer Science and Artificial Intelligence, Zhengzhou University, Zhengzhou, China. His research interests include parallel and distributed computing, embedded and ubiquitous/pervasive computing, and cyber–physical–social systems.



Xinfang Sun received the B.E. degree in English from Xinyang Normal University, Xinyang, China, in 2009, and the B.Sc. degree in business administration from the Henan University of Economics and Law, Zhengzhou, China, in 2024.

She is currently a Lecturer with Zhengzhou Normal University, Zhengzhou. From 2010 to 2025, she served as a Department Manager with New China Life Insurance Co., Ltd., focusing on financial asset risk management and related areas. Her research interests include artificial intelligence security and blockchain.



Zhong Chen received the B.S. and Ph.D. degrees in software engineering from Peking University, Beijing, China, in 1983 and 1989, respectively.

He is currently a Professor with Peking University. He currently serves as the Director of the Key Laboratory of Network and Software Security Assurance of Chinese Ministry of Education. He has published more than 200 scientific papers, including ICSE, ISSTA, ICPC, etc. His research interests include cryptography and blockchain.

Authorized licensed use limited to: James Cook University of Northern Queensland. Downloaded on October 10,2026 at 04:01:53 UTC from IEEE Xplore. Restrictions apply.